



Test Automation Framework (TAF::Perl)

Overview Document

MariaDB Foundation <https://mariadb.org>

Open, vendor-neutral, community-driven tooling for database testing and benchmarking.

TAF-Perl is maintained and published by the MariaDB Foundation to support transparent, reproducible, and fair performance evaluation across the open source database ecosystem. The framework is part of the Foundation's mission to provide open tooling that benefits the entire database community.

© MariaDB Foundation. All rights reserved.

Introduction

TAF-Perl is a deterministic and extensible automation framework designed for running client-to-backend test suites, benchmarking workloads, and validating database systems with reproducible, contributor-proof behavior. It provides a stable foundation for performance testing and system verification across multiple database makers while remaining lightweight, explicit, and free of external dependencies.

The framework exists to fill a long-standing gap in the open database ecosystem: a vendor-neutral and community-driven test framework capable of supporting MariaDB, MySQL, and any other database engine for which a TAF database plugin, SQL dialect, and compatible test client are available. Unlike proprietary or vendor-locked tools, TAF-Perl is built to serve the broader community and to ensure that testing workflows remain open, transparent, and reproducible.

TAF-Perl continues the architectural lineage of the Autobench family of tools, inheriting the proven lifecycle patterns and operational discipline that made Autobench effective. However, it is not a fork of Autobench3 (AB3) and shares no code with it. AB3 is an Oracle-owned, MySQL-only, XML-driven framework used internally for automated farm runs and regression qualification. It was never designed to support MariaDB or any other database maker. TAF-Perl extends the lineage into a fully vendor-neutral space, enabling consistent testing across multiple engines through a clean plugin architecture.

A central goal of TAF-Perl is to strengthen the open source database ecosystem by providing a transparent, repeatable, and community-maintained framework for performance evaluation. By making the tooling open and accessible, the MariaDB Foundation aims to encourage more engineers, researchers, and practitioners to benchmark databases fairly, compare systems on equal footing, and contribute improvements that benefit the entire community. TAF-Perl is designed to lower the barrier to entry for benchmarking and to promote a culture of reproducible, evidence-based performance analysis across the industry.

At its core, TAF-Perl is built around deterministic execution, explicit state transitions, and contributor-proof design. Every action, lifecycle step, and plugin interface is structured to ensure predictable behavior, reproducibility, and clarity for both users and contributors. This design philosophy aligns with the mission of the MariaDB Foundation: to provide open, community-driven tooling that strengthens the long-term health of the database ecosystem.

TAF-Perl supports MariaDB, MySQL, and any additional database maker for which a plugin and SQL dialect are implemented. This extensibility ensures that the framework can evolve with the ecosystem while maintaining its core principles of neutrality, determinism, and operational simplicity.

Architectural Identity

TAF::Perl is built on a set of architectural principles that ensure predictable behavior, long-term maintainability, and transparent operation across all supported environments. These principles define how the framework behaves internally and how contributors can reason about its execution model.

At the core of the framework is deterministic execution. Every lifecycle step, plugin invocation, and state transition follows a clearly defined sequence, eliminating ambiguity and preventing hidden or implicit behavior. This determinism ensures that test results are reproducible across machines, environments, and contributors, which is essential for meaningful performance evaluation.

The framework operates through an explicit lifecycle. Test suites, database plugins, and reporters each implement well-defined routines that TAF::Perl executes in a fixed order. Nothing occurs implicitly, and no component is allowed to introduce side effects outside its designated lifecycle boundaries. This explicit structure makes the framework easy to inspect, reason about, and extend.

TAF::Perl requires no external dependencies. It runs entirely on the Perl interpreter already present on all major Linux distributions used for database development and testing. There are no virtual environments, package managers, or external modules to install. This zero-dependency model ensures that the framework behaves consistently on bare systems, CI hosts, containers, and remote test machines.

Because it runs anywhere Perl exists, TAF::Perl is inherently portable. The same test suite, plugin, or configuration can be executed on any supported platform without modification. This portability reinforces the framework's commitment to reproducibility and makes it suitable for distributed testing environments.

Contributor-proof behavior is a defining characteristic of the framework. All state transitions, file operations, and lifecycle routines are explicit and visible. There are no hidden defaults, no implicit environment assumptions, and no silent fallbacks. This design prevents accidental misuse, reduces onboarding friction, and ensures that contributors can extend the framework without introducing ambiguity.

These architectural principles align directly with the mission of the MariaDB Foundation. By providing a deterministic, transparent, and vendor-neutral automation framework, the Foundation supports fair benchmarking, reproducible testing, and open collaboration across the database ecosystem. TAF::Perl embodies this mission by offering a stable and community-maintained foundation for evaluating database performance and behavior.

(Continue next page)

Plugin Architecture

TAF::Perl is built around a clean, modular plugin architecture that separates database behavior, test suite logic, and reporting into independent components. This design ensures that each part of the framework remains deterministic, maintainable, and easy to extend. Plugins operate under explicit lifecycle rules and never introduce hidden behavior, preserving the contributor-proof nature of the framework.

Database Plugins

Database plugins define how TAF::Perl interacts with a specific database maker. When a test run begins, TAF resolves the active database software installation and loads the corresponding plugin. If the user explicitly requests a plugin, TAF verifies that the requested maker matches the active installation. Mismatches are rejected to prevent accidental cross-maker execution.

Each database plugin is responsible for implementing the full backend lifecycle, including installation, initialization, startup, shutdown, and validation routines. These routines encapsulate all maker-specific behavior behind a stable API, ensuring that test suites and tools never need to know the details of how a particular database is installed or managed.

The plugin model is fully vendor-neutral. Any database maker can be supported as long as a plugin, SQL dialect, and compatible test client exist. This neutrality ensures that TAF::Perl remains open to the entire database ecosystem and avoids the vendor-locked behavior found in proprietary frameworks.

Test Suite Plugins

Test suites in TAF::Perl are implemented as plugins that define a clear and deterministic lifecycle. Each suite may implement routines such as PreTestSetup, TestSetup, TestRun, TestPost, and TestCleanup. These routines are executed in a fixed order, ensuring that all test runs follow the same predictable structure.

In addition to lifecycle routines, test suites may provide optional metadata and control routines. These include methods for defining default tests, legal test names, thread counts, duration, version information, and client build requirements. Suites may also specify SQL dialect requirements, allowing TAF::Perl to validate compatibility with the active database software installation and plugin before execution begins.

Because test suites operate under the same explicit lifecycle rules as database plugins, they remain easy to inspect, reason about, and extend. No suite can introduce implicit behavior or override lifecycle order, preserving the deterministic nature of the framework.

Reporter Plugins

Reporter plugins are independent modules responsible for generating output from test results. They do not influence test execution and have no effect on the behavior of database or test suite plugins. Instead, they receive structured results from the framework and produce output in one or more formats.

Reporters can be stacked, allowing multiple output formats to be generated from a single test run. Common reporter types include JSON, text, HTML, and archival formats. Because each reporter operates independently, users can customize their reporting pipeline without modifying test suites or database plugins.

This multi-format output pipeline ensures that TAF::Perl can integrate cleanly with dashboards, CI systems, archival workflows, and external analysis tools. The reporter model reinforces the framework's commitment to modularity and vendor-neutral design.

TAF Directory Structure

The TAF directory layout is intentionally structured to keep execution, configuration, reporting, and environment management cleanly separated. Each top-level directory serves a specific operational role in the framework, ensuring that test runs remain reproducible, isolated, and easy to debug. The table below provides a concise description of what each directory contains and why it exists, giving new users and contributors a clear understanding of how the framework is organized.

Directory	Description
archive/	Stores compressed logs, results, and artifacts from completed test runs.
client_source/	Contains source code for client-side tools (e.g., <code>sysbench-lua</code>).
data/	Holds generated datasets and temporary files used during execution.
database_config_files/	Contains DB-specific config templates for various engines.
database_software_installs/	Local install trees for MySQL, MariaDB, PG, etc. — used for isolated runs.
external_tools/	Third-party binaries or kits (e.g., BMK) integrated into the harness.
help/	CLI help files and usage documentation.
libs/	Core Perl modules that define TAF behavior and orchestration.
logs/	Runtime logs from active or completed executions.
properties/	Default and user-defined property files for suite configuration.
reports/	Generated reports from completed runs.
results/	Raw results and metrics from test executions.
scripts/	Utility scripts and wrappers used by TAF.
test_suites/	Contains all defined test suites and workload definitions.
tmp/	Temporary working directory for intermediate files.

(Continue next page)

TAF Core Modules

The TAF Core Modules make up the internal engine that powers `taf.pl`. These modules define the framework's behavior, lifecycle, configuration handling, database orchestration, logging, reporting, and test-suite management. Together, they form the foundation of the Test Automation Framework, providing the structure and functionality that every test run depends on.

File/Module	Description
<code>ActionWrappers.pm</code>	TAF runs off of actions. This action wrapper implements all TAF's actions. See action list.
<code>Archive.pm</code>	Handles archiving logs, sub-results directories, and reports from a test run.
<code>Client.pm</code>	Handles client builds using CMake, e.g. sysbench-lua.
<code>CommandLine.pm</code>	Processes command-line arguments.
<code>Database.pm</code>	Handles loading and communicating with database maker plugins.
<code>DatabaseSoftwareInstalls.pm</code>	Handles database software installation and related setup.
<code>Logging.pm</code>	Wrapper around the utility <code>Logger.pm</code> to make logging useful to the framework.
<code>Properties.pm</code>	Handles processing of properties.
<code>Reports.pm</code>	Interface to report plugins.
<code>Run.pm</code>	Handles test execution.
<code>TestSuiteManagement.pm</code>	Handles loading and working with test suite(s).
<code>Utilities.pm</code>	Provides utility functions to the framework.

(Continue next page)

TAF Database Plugin Modules

The TAF Database Plugin Modules define the contract that makes cross-engine comparison possible.

Each database implementation—MySQL, MariaDB, or any future engine—conforms to the same lifecycle, the same actions, and the same expectations. By standardizing install, init, start, stop, cleanup, and configuration behaviors behind a shared plugin interface, TAF can run identical workloads across different database systems without special-case logic or one-off scripts. This is the foundation that allows true apples-to-apples benchmarking and makes expanding support for new engines straightforward.

File/Plugin	Description
<code>db_plugin_template.pm</code>	Template plugin showing the required structure and functions needed to implement a new database maker plugin.
<code>mariadb.pm</code>	Database plugin for MariaDB. Implements all required lifecycle actions (install, init, start, stop, cleanup) using MariaDB-specific logic.
<code>mysql.pm</code>	Database plugin for MySQL. Implements the full database lifecycle using MySQL-specific behaviors and configuration rules.

(Continue next page)

TAF Reporter Plugins Overview

The TAF Reporter Plugins provide the final translation layer between raw test output and human- or machine-readable results. Each plugin focuses on a specific reporting format or visualization style, allowing TAF to present the same underlying data in HTML, text, CSV, JSON, or chart-driven layouts without changing the test flow itself. This modular design keeps reporting flexible, extensible, and consistent across engines and workloads, making it easy to plug in new formats or integrate with external systems.

File/Plugin	Description
<code>taf_paths.pm</code>	Provides path helpers and shared directory-location logic used by reporter plugins to find logs, results, and artifacts.
<code>chart_and_tables_html.pm</code>	Generates HTML output containing charts and tabular result summaries.
<code>chart_and_test_info_results_tables_html.pm</code>	Generates combined HTML output including charts, test-info blocks, and results tables. (Extended version of <code>chart_and_tables_html</code> .)
<code>csv.pm</code>	Produces CSV -formatted output for results, suitable for spreadsheets or downstream tooling.
<code>json.pm</code>	Produces JSON -formatted output for results, enabling machine-readable consumption and integration with external systems.
<code>taf_res_raw_text.pm</code>	Outputs raw text versions of results, useful for debugging or simple command-line consumption.
<code>text.pm</code>	Generates human-readable plain-text reports summarizing test results.
<code>troch_queries_chart.pm</code>	Generates charts specifically for TROC/H query workloads or query-focused result sets.

(Continue next page)

TAF Utility Modules

The TAF Utility Modules provide the low-level building blocks that support scripting, file handling, environment detection, packaging, and property processing across the framework. These helpers keep the higher-level systems clean by centralizing common operations into small, focused modules.

Module Name	Category	Description
<code>toolsLib.pm</code>	Core Infrastructure	Central entry point that exposes many of the helper utilities in one place.
<code>Logger.pm</code>	Core Infrastructure	Basic logging utilities used by scripts and helper modules.
<code>Archiver.pm</code>	Build & Packaging	Archive utilities for packaging logs, results, and other artifacts.
<code>ClientCmakeBuild.pm</code>	Build & Packaging	Utilities for building client tools using CMake.
<code>Extractor.pm</code>	Build & Packaging	Utilities for unpacking and extracting archive files.
<code>InstallSearch.pm</code>	Build & Packaging	Search utilities used to locate installed components or paths.
<code>FileOps.pm</code>	File System	Basic file-operation utilities (copy, move, delete, etc.).
<code>Paths.pm</code>	File System	Path-handling utilities for normalizing and constructing filesystem paths.
<code>RemoveDir.pm</code>	File System	Directory-cleanup utilities for safe and recursive removal.
<code>FileCount.pm</code>	File System	Utilities for counting files and directories.
<code>SecureCopy.pm</code>	File System	SCP utilities for secure file transfer operations.
<code>SystemInfo.pm</code>	System & Env	Utilities for gathering system information and environment details.
<code>GetHostName.pm</code>	System & Env	Utilities for retrieving and handling hostnames.
<code>DateTime.pm</code>	System & Env	Date and time helper utilities.
<code>Properties.pm</code>	System & Env	Utilities for processing and managing property files.
<code>PropertiesParser.pm</code>	Utilities	Parser that invokes <code>Properties.pm</code> to load and interpret property data.
<code>IsNumeric.pm</code>	Utilities	Utilities for numeric validation and checks.
<code>Switch.pm</code>	Utilities	Provides simple switch-statement-style logic for scripts.
<code>Trim.pm</code>	Utilities	Text-trimming utilities for cleaning whitespace and formatting.

TAF SQL Executor

The TAF SQL Executor provides a unified way to run SQL across multiple database engines without forcing test authors to rewrite or maintain engine-specific scripts. By separating SQL logic from SQL dialect, TAF ensures that the same logical operations can be executed on MySQL, MariaDB, PostgreSQL, Oracle, or any future engine simply by selecting the appropriate dialect file. The Executor applies the correct syntax, formatting, and engine-specific adjustments automatically, allowing test suites to remain portable, consistent, and comparable across environments. This abstraction is what makes true cross-database benchmarking possible: one test flow, one set of SQL operations, many engines — all normalized through a single execution layer.

Module / File	Category	Description
Executor.pm	Core Engine	Executes SQL files using the selected database plugin and applies the correct dialect automatically.
dialects/mariadb.sql	SQL Dialects	SQL definitions tailored for MariaDB's syntax and behavior.
dialects/mysql.sql	SQL Dialects	SQL definitions tailored for MySQL's syntax and behavior.
dialects/oracle.sql	SQL Dialects	SQL definitions tailored for Oracle's SQL dialect.
dialects/postgres.sql	SQL Dialects	SQL definitions tailored for PostgreSQL's SQL dialect.

Example:

```
sql

# -----
# Example SQL Dialect Definitions
# -----
# File: dialects/example.sql
# Purpose:
#   Demonstrates the structure and conventions used by TAF SQL dialect
#   files. Each block defines a named SQL snippet that TAF can load
#   and execute through Executor.pm.
#
# Copyright:
#   Copyright (c) Example.
#   Licensed under the Apache License, Version 2.0.
# -----

[version]
SELECT VERSION();

[variables]
SHOW VARIABLES;

[row_count]
SELECT COUNT(*) FROM {table};

[db_size]
SELECT table_schema AS db,
       SUM(data_length + index_length) AS size_bytes
FROM information_schema.tables
GROUP BY table_schema;

[stats]
SHOW GLOBAL STATUS;

[create_database]
CREATE DATABASE {db};

[drop_database]
DROP DATABASE IF EXISTS {db};
```

TAF Test Suite Modules

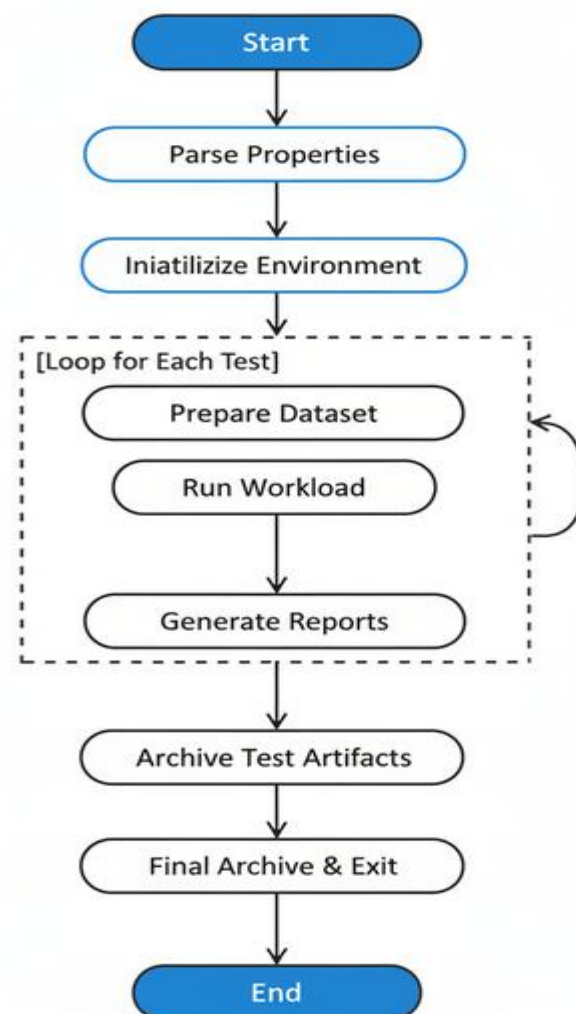
The Test Suite Modules are the heart of TAF. Everything else in the framework exists to support what happens here. Without test suites, there is no workload, no metrics, no comparison, no story to tell. These modules define the actual benchmark logic — the transactions, the queries, the load patterns, the runtime behavior — that TAF executes across different database engines. By implementing each workload behind a consistent interface, TAF can run HammerDB, Sysbench-Lua, or any future benchmark with the same lifecycle, the same reporting pipeline, and the same cross-engine comparability. This is the meat of the system: the place where real performance is measured, where engines are pushed, and where results become meaningful.

Module Name	Category	Description
hammerdb-tprocc.pm	Test Suites	Implements the HammerDB TPROC-C workload as a TAF test suite.
hammerdb-tproch.pm	Test Suites	Implements the HammerDB TPROC-H workload as a TAF test suite.
sysbench-lua.pm	Test Suites	Implements Sysbench-Lua workloads using the TAF test suite API.
test_suite_template.p m	Test Suites	Template showing the required structure and lifecycle methods for creating new TAF test suites.

(Continue next page)

TAF Execution Workflow

The TAF Execution Workflow defines the core lifecycle that every test run follows. This is the operational backbone of the entire framework: the sequence that turns a simple properties file into a fully executed, fully archived, fully reported benchmark run. Each stage is explicit, deterministic, and contributor-proof, ensuring that no matter how many tests are defined, each one is executed cleanly, isolated from the others, and archived with its own results and artifacts. The loop at the center of the workflow is where the real work happens — dataset preparation, workload execution, result harvesting, and reporting — repeated for every test defined by the user. Without this structure, TAF would be a loose collection of components. With it, TAF becomes a reliable, repeatable, multi-test benchmarking engine.



Database Software Install Model

TAF::Perl includes a built-in model for managing database software installations in a deterministic and vendor-neutral way. The framework can install database software, track multiple installations, validate them, and designate one as the active installation for all subsequent actions. This model ensures that test runs are always executed against a known, explicit, and reproducible backend environment.

Supported Package Types

TAF::Perl supports a wide range of installation formats to accommodate the packaging practices of different database makers. Supported package types include:

- tar.gz archives
- RPM packages
- DEB packages
- zipped source bundles
- vendor-specific formats, when supported by the corresponding database plugin

This flexibility allows TAF::Perl to install and manage database software across diverse environments without requiring custom tooling or external dependencies.

Install Management

The install management system provides a consistent interface for handling database software across makers and platforms. TAF::Perl can install new database software, list existing installations, validate their structure, and switch between them as needed.

Users may perform the following actions:

- Install database software from one or more packages
- List all known database software installations
- Set the active installation used for all TAF actions
- Validate the structure and completeness of an installation

The concept of the active install is central to the framework. All TAF actions, including setup-client, setup-db, start-db, and run-tests, operate against the active installation. When a user selects or installs a new database version, TAF::Perl updates the active install marker to ensure that all subsequent operations use the correct backend. This contract prevents accidental cross-version or cross-maker execution and reinforces the deterministic behavior of the framework.

Relevant Command Options (High-Level Only)

The following command options provide access to the install management system. These descriptions are intentionally high-level; full details are provided in the Database Software Install Quick Start guide.

--db-software-install

- Perform a direct database software install without requiring a properties file or TAF action.

--list-database-software-installs

- List all known database software installs.

--list-active-db-install

- Show the active database software install.

--set-active-database-software-install

- Mark a database software install as active.

--db-software-install-packages=/path/pkg1.tar.gz[,pkg2.rpm,...]

- Packages to install.

--db-software-install-dir=<path>

- Directory of an existing database software install. Using this option also updates the active install marker, making the specified directory the active installation for all subsequent TAF actions (setup-client, setup-db, start-db, run-tests, etc.).

(Continue next page)

TAF Lifecycle Flow

TAF::Perl follows a strict and deterministic lifecycle that governs every test run. The lifecycle begins before any test suite or database plugin is loaded, ensuring that configuration, overrides, and early-exit operations are handled consistently and predictably. This explicit flow prevents ambiguity, eliminates hidden behavior, and guarantees reproducibility across environments and contributors.

The lifecycle begins with early command-line processing. TAF::Perl first inspects the command-line arguments to determine whether the user has requested an informational or management action such as listing database installs, setting the active install, printing help, or performing file-management operations. These actions are handled immediately and outside the main test lifecycle. If an early-exit action is triggered, the framework performs the requested operation and terminates cleanly.

If no early-exit action is requested, the framework proceeds to initialize configuration. Default properties are loaded first, establishing the baseline configuration for the run. A temporary options structure is then created, and the command-line arguments are parsed into this temporary override layer. These temporary overrides allow TAF::Perl to resolve early decisions—such as which properties file to load—without committing final overrides prematurely.

Next, the user properties file is loaded. User properties override all default properties and form the primary configuration for the test run. After user properties are merged, the command-line arguments are applied a second time, this time as final overrides. This two-stage override model ensures that command-line options always take precedence while still allowing early decisions to be made deterministically.

Once configuration is finalized, TAF::Perl resolves the active database software installation. If the user has explicitly selected an installation, the framework validates it and ensures that it matches the requested database maker. If no selection is provided, the active install marker determines which backend will be used. This step ensures that all subsequent actions operate against a known and validated installation.

With configuration and installation resolved, the framework loads the database plugin associated with the selected maker. The plugin provides backend lifecycle routines, SQL dialect mapping, validation logic, and optional client build routines. This abstraction allows TAF::Perl to support multiple database makers through a consistent and vendor-neutral interface.

The test suite plugin is then loaded. The suite defines the lifecycle routines that drive the test itself, including setup, execution, post-processing, and cleanup. Optional metadata routines may also be invoked to determine thread counts, duration, legal test names, or client build requirements. The deterministic ordering of these routines ensures that every test run follows the same structure.

Reporter plugins are loaded next. These modules receive structured results from the test run and generate output in one or more formats. Reporters do not influence execution; they operate strictly after the test suite produces results.

With all components loaded, TAF::Perl executes the lifecycle routines in a fixed, deterministic order. Each routine is invoked explicitly, and no plugin is permitted to alter the sequence. This guarantees that test runs remain reproducible across machines, environments, and contributors.

After execution completes, the framework produces reports through the configured reporter plugins. These may include text summaries, JSON output, HTML reports, or archival bundles. The reporting pipeline is modular and stackable, allowing multiple formats to be generated from a single run.

Finally, TAF::Perl archives the results. Logs, reports, metadata, and artifacts are stored in a structured

directory layout to support long-term reproducibility and external analysis. The archive step ensures that every run is preserved in a consistent and inspectable format.

This lifecycle flow is central to the identity of TAF::Perl. By enforcing a deterministic, reproducible sequence of operations—from early command-line handling through final archival—the framework provides a stable foundation for fair benchmarking, regression testing, and system validation across the open source database ecosystem.

(Continue next page)

Philosophy and Design Principles

TAF::Perl is shaped by a set of core design principles that guide every architectural decision, lifecycle rule, and contributor workflow within the framework. These principles ensure that the system remains predictable, maintainable, and aligned with the values of the open source database community. Deterministic behavior is the foundation of the framework. Every lifecycle step, plugin invocation, and configuration override follows a clearly defined sequence, eliminating ambiguity and preventing unexpected behavior. Determinism ensures that test results are reproducible across machines, environments, and contributors, which is essential for meaningful benchmarking and regression analysis.

Explicit state transitions reinforce this determinism. TAF::Perl does not allow hidden state, implicit defaults, or silent fallbacks. All state changes — from configuration merging to backend selection to lifecycle execution — are visible, intentional, and traceable. This explicitness makes the framework easy to reason about and prevents accidental misuse.

Contributor-proof workflows are a defining characteristic of the system. The architecture is designed so that new contributors cannot accidentally break invariants, introduce hidden behavior, or bypass lifecycle rules. Clear contracts, explicit routines, and strict ordering ensure that extensions remain safe and predictable, even as the framework evolves.

Vendor neutrality is central to the mission of TAF::Perl. The framework does not privilege any database maker and does not embed vendor-specific assumptions. Database behavior is encapsulated entirely within plugins, allowing any maker to be supported as long as a plugin, SQL dialect, and test client exist. This neutrality enables fair comparisons and prevents vendor lock-in.

Long-term maintainability is achieved through simplicity, clarity, and architectural discipline. The framework avoids unnecessary abstraction, minimizes global state, and uses well-defined interfaces for all plugins and lifecycle routines. This approach ensures that TAF::Perl remains approachable for future contributors and stable across years of evolution.

The zero-dependency operational model further strengthens maintainability. TAF::Perl runs on the Perl interpreter already present on all major Linux distributions used for database development and testing. No external modules, package managers, or network access are required. This model guarantees consistent behavior across environments and eliminates dependency drift.

These principles align directly with the mission of the MariaDB Foundation: to provide open, community-driven tooling that strengthens the database ecosystem. By emphasizing determinism, transparency, neutrality, and long-term stability, TAF::Perl supports fair benchmarking, reproducible testing, and collaborative engineering across the open source community.

(Continue next page)

Closing Summary

TAF-Perl provides a deterministic, extensible, and vendor-neutral foundation for database testing, benchmarking, and system validation. By combining explicit lifecycle rules, a clean plugin architecture, and a zero-dependency operational model, the framework delivers a level of clarity and reproducibility that is essential for meaningful performance evaluation. It enables engineers to run consistent workloads across multiple database makers without hidden behavior, implicit assumptions, or environment-specific drift.

The framework matters because fair benchmarking and reproducible testing are increasingly important in a diverse and rapidly evolving database ecosystem. TAF-Perl lowers the barrier to entry for performance analysis, encourages transparent engineering practices, and supports evidence-based comparisons across vendors. Its deterministic design ensures that results can be trusted, shared, and reproduced by anyone in the community.

Within the MariaDB ecosystem, TAF-Perl serves as a critical piece of open infrastructure. It provides a stable and community-maintained platform for evaluating MariaDB performance, validating new releases, and supporting long-term engineering workflows. At the same time, its vendor-neutral architecture ensures that it remains useful beyond any single database maker, reinforcing the Foundation's commitment to openness and collaboration.

TAF-Perl is built to be future-proof. Its plugin-driven architecture allows new database makers, SQL dialects, test suites, and reporting formats to be added without disrupting existing workflows. Its contributor-proof design ensures that extensions remain safe, predictable, and maintainable. As the database ecosystem continues to evolve, TAF-Perl provides a stable, extensible, and community-driven framework that will continue to support fair benchmarking and reproducible testing for years to come.

End of Document

This overview was prepared and published by the MariaDB Foundation <https://mariadb.org>

TAF::Perl is part of the Foundation's commitment to open, vendor-neutral tooling for the database ecosystem. The framework is maintained to support fair benchmarking, reproducible testing, and transparent engineering practices across all database makers.

For contributions, feedback, or questions, please visit the Foundation website or contact the maintainers directly.

© MariaDB Foundation. All rights reserved.